COMPTE RENDU TP3

EX2:

```
= RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py
[D1] texte original : Information retrieval systems use inverted indexes for eff
icient search
     tokens              : ['information', 'retrieval', 'systems', 'use', 'inver
ted', 'indexes', 'for', 'efficient', 'search']
     sans mots vides     : ['information', 'retrieval', 'systems', 'use', 'inver
ted', 'indexes', 'efficient', 'search']
     termes indexés      : ['information', 'retrieval', 'system', 'use', 'invert
', 'index', 'efficient', 'search']

[D4] texte original : TF IDF measures the importance of terms in documents
     tokens              : ['tf', 'idf', 'measures', 'the', 'importance', 'of',
'terms', 'in', 'documents']
     sans mots vides     : ['tf', 'idf', 'measures', 'importance', 'terms', 'doc
uments']
     termes indexés      : ['tf', 'idf', 'measure', 'importance', 'term', 'docum
ent']

[D7] texte original : Neural information retrieval uses deep learning models
     tokens              : ['neural', 'information', 'retrieval', 'uses', 'deep'
, 'learning', 'models']
     sans mots vides     : ['neural', 'information', 'retrieval', 'uses', 'deep'
, 'learning', 'models']
     termes indexés      : ['neural', 'information', 'retrieval', 'use', 'deep',
 'learn', 'model']


============== RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py ==============
```

reponse question: il faut appliquer exactement le même pipeline de prétraitement aux documents et aux requêtes car la recherche consiste à comparer des termes de la requête avec des termes de l'index

EX3:

```
= RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py
[D1] texte original : Information retrieval systems use inverted indexes for eff
icient search
     tokens              : ['information', 'retrieval', 'systems', 'use', 'inver
ted', 'indexes', 'for', 'efficient', 'search']
     sans mots vides     : ['information', 'retrieval', 'systems', 'use', 'inver
ted', 'indexes', 'efficient', 'search']
     termes indexés      : ['information', 'retrieval', 'system', 'use', 'invert
', 'index', 'efficient', 'search']

[D4] texte original : TF IDF measures the importance of terms in documents
     tokens              : ['tf', 'idf', 'measures', 'the', 'importance', 'of',
'terms', 'in', 'documents']
     sans mots vides     : ['tf', 'idf', 'measures', 'importance', 'terms', 'doc
uments']
     termes indexés      : ['tf', 'idf', 'measure', 'importance', 'term', 'docum
ent']

[D7] texte original : Neural information retrieval uses deep learning models
     tokens              : ['neural', 'information', 'retrieval', 'uses', 'deep'
, 'learning', 'models']
     sans mots vides     : ['neural', 'information', 'retrieval', 'uses', 'deep'
, 'learning', 'models']
     termes indexés      : ['neural', 'information', 'retrieval', 'use', 'deep',
 'learn', 'model']
```

"""TF(t,d) (term frequency) est le nombre d'occurrences du terme t dans un document précis d

DF(t) (document frequency) est le nombre de documents de la collection qui contiennent le terme t"""

EX4:

Terme	DF	IDF
information	5	0.6990
retrieval	11	0.3565
vector	4	0.7959
boolean	3	0.9208

DF max : retrieval (DF=11)

DF min : systems (DF=1) [premier terme rencontré à DF minimal]

IDF max : systems (IDF=1.3979)

Parce que l'IDF est conçu pour mesurer le **pouvoir discriminant** d'un terme, et qu'un terme rare discrimine beaucoup plus qu'un terme répandu.

EX5 :

D13 : « Cosine similarity compares the angle between a query vector and a document vector »

Terme	**TF**	**DF**	**IDF**	**TF-IDF**
cosine	1	1	1,3979	**1,3979**
compares	1	1	1,3979	**1,3979**
angle	1	1	1,3979	**1,3979**
vector	**2**	4	0,7959	1,0355
similarity	1	4	0,7959	0,7959
document	1	4	0,7959	0,7959
query	1	5	0,6990	0,6990

vector a le plus grand TF (2), mais seulement le 4e poids TF-IDF. Il est dépassé par cosine, compares et angle, qui n'apparaissent qu'une fois dans D13 mais dans aucun autre document. Leur IDF élevé (1,3979) compense leur TF de 1.

EX6:

```
= RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py
D1 : 92 composantes, 84 nulles (91.3 %)
   vecteur complet : [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
, 0.0, 0.0, 0.0, 0.0, 0.0, 1.398, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.796, 0.0, 0.699, 0.0, 0.921, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.357, 0.0, 0.0, 0.0, 0.495, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 1.398, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.699, 0.0, 0.0, 0.0, 0.
0, 0.0, 0.0, 0.0]
   non nulles      : {'efficient': 1.398, 'index': 0.796, 'information': 0.699,
'invert': 0.921, 'retrieval': 0.357, 'search': 0.495, 'system': 1.398, 'use': 0.
699}

D2 : 92 composantes, 86 nulles (93.5 %)
   vecteur complet : [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0
, 0.0, 0.0, 0.0, 0.0, 0.222, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.699, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.495, 0.
0, 0.0, 0.0, 0.0, 0.0, 1.097, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.398, 0.0, 0.0
, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.035, 0
.0, 0.0, 0.0]
   non nulles      : {'document': 0.222, 'model': 0.699, 'query': 0.495, 'repres
ent': 1.097, 'space': 1.398, 'vector': 1.035}

D3 : 92 composantes, 88 nulles (95.7 %)
   vecteur complet : [0.0, 0.0, 0.921, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0
.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.398, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
 0.0, 0.0, 0.0, 0.0, 0.0, 0.357, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.
0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.699, 0.0, 0.0, 0.0, 0.0, 0.0,
0.0, 0.0]
   non nulles      : {'boolean': 0.921, 'operator': 1.398, 'retrieval': 0.357, '
use': 0.699}
```

Chaque vecteur a **une composante par terme du vocabulaire** de toute la collection (105 ici). Or un document ne contient qu'une **petite fraction** de ces termes : D3 n'en contient que 4 sur 105. Tous les autres termes valent 0, car TF(t,d) = 0 implique w(t,d) = 0

EX7:

```
= RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py
Requête : information retrieval model
TERM           TF   IDF      POIDS
information    1    0.6990   0.6990
model          1    0.6990   0.6990
retrieval      1    0.3565   0.3565
```

Le programme doit **ignorer ce terme**, sans produire d'erreur.

EX8:

```
= RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py
Requête : information retrieval model

DocID  Score
D1     0.2257
D2     0.2077
D3     0.0654
D4     0.0000
D5     0.0000
D6     0.2556
D7     0.3849
D8     0.0538
D9     0.0000
D10    0.0000
D11    0.0000
D12    0.1513
D13    0.0000
D14    0.0000
D15    0.0434
D16    0.1721
D17    0.2552
D18    0.0000
D19    0.3075
D20    0.0371
D21    0.0000
D22    0.0000
D23    0.0000
D24    0.0387
D25    0.1966

Test vecteur nul : 0.0
```

- **Proche de 1** : les deux vecteurs pointent dans la même direction. La requête et le document utilisent les mêmes termes, avec des poids proportionnels : le document est très pertinent.
- **Proche de 0** : les vecteurs sont presque perpendiculaires. Ils ont peu ou pas de termes en commun : le document est peu ou pas pertinent. Un score de **exactement 0** (D4, D5, D9, etc.) signifie qu'aucun terme de la requête n'apparaît dans le document.

EX9:

========== RESTART: C:/Users/Lenovo/Desktop/tp index/tp3/tp3.py ==========

Test vecteur nul : 0.0

Query > information retrieval model

TOP-5 RESULTS

Rank DocID Score Document

---

1 D7 0.3849 Neural information retrieval uses deep learning models

2 D19 0.3075 The vector model ranks documents by similarity with the query unlike the Boolean model

3 D6 0.2556 BM25 is a ranking function used in information retrieval

4 D17 0.2552 Information retrieval evaluates the relevance of documents for a user query

5 D1 0.2257 Information retrieval systems use inverted indexes for efficient search

Query > dense retrieval embeddings

TOP-5 RESULTS

Rank DocID Score Document

---

1 D8 0.5638 Dense retrieval represents queries and documents using embeddings

2 D15 0.4551 Dense retrieval encodes documents into embeddings and searches nearest neighbors

3 D24 0.2460 Hybrid search combines BM25 and dense retrieval to improve ranking quality

4 D9 0.2030 Vector databases support similarity search over embeddings

5 D22 0.1833 Embeddings capture semantic similarity between words queries and documents

Query > inverted index search engines

TOP-5 RESULTS

Rank DocID Score Document

---

1 D10 0.5888 Search engines use inverted indexes to retrieve documents efficiently

2 D1 0.4329 Information retrieval systems use inverted indexes for efficient search

3 D11 0.3435 An inverted index maps each term to the list of documents containing it

4 D20 0.3014 Web search engines crawl pages then build an index for fast retrieval

5 D5 0.2632 Search engines rank documents according to their relevance

Query > boolean model operators

TOP-5 RESULTS

Rank DocID Score Document

---

1 D3 0.8355 Boolean retrieval uses AND OR and NOT operators

2 D19 0.4158 The vector model ranks documents by similarity with the query unlike the Boolean model

3 D12 0.2397 The Boolean model returns documents that exactly match the query expression

4 D2 0.1203 The vector space model represents documents and queries as vectors

5 D7 0.0986 Neural information retrieval uses deep learning models

Query > bm25 ranking function

TOP-5 RESULTS

Rank DocID Score Document

---

1 D6 0.8086 BM25 is a ranking function used in information retrieval

2 D24 0.2557 Hybrid search combines BM25 and dense retrieval to improve ranking quality

3 D19 0.1737 The vector model ranks documents by similarity with the query unlike the Boolean model

4 D5 0.1573 Search engines rank documents according to their relevance

5 D14 0.1337 BM25 improves TF IDF with term frequency saturation and document length normalization

EX10:

Mesure Sans stemming Avec stemming

Score doc contenant model (D2) 0.2548 0.2077

Score doc contenant models (D7) 0.1743 0.3849

Rang doc contenant model (D2) 2 6

Rang doc contenant models (D7) 7 1

Top-5 sans stemming : [('D19', 0.369), ('D2', 0.255), ('D12', 0.22), ('D6', 0.211), ('D17', 0.202)]

Top-5 avec stemming : [('D7', 0.385), ('D19', 0.308), ('D6', 0.256), ('D17', 0.255), ('D1', 0.226)]

EX11:

Requête : information retrieval model

Booléen AND : aucun document (aucun ne contient les 3 termes)

Vectoriel : [('D19', 0.369), ('D2', 0.255), ('D12', 0.22), ('D6', 0.211), ('D17', 0.202)]

Termes de la requête présents dans chaque doc du Top-5 : [('D19', ['model']), ('D2', ['model']), ('D12', ['model']), ('D6', ['information', 'retrieval']), ('D17', ['information', 'retrieval'])]

Requête : dense information retrieval

Booléen AND : aucun document (aucun ne contient les 3 termes)

Vectoriel : [('D8', 0.336), ('D15', 0.261), ('D24', 0.241), ('D6', 0.211), ('D17', 0.202)]

Termes de la requête présents dans chaque doc du Top-5 : [('D8', ['dense', 'retrieval']), ('D15', ['dense', 'retrieval']), ('D24', ['dense', 'retrieval']), ('D6', ['information', 'retrieval']), ('D17', ['information', 'retrieval'])]